

ME 403 – Introduction to Robotics

Term Project Report

Unitree Z1 Robot Arm: Kinematics, Dynamics, and Control

Student Names:

Arda Barış Tonbil

Elest Cemre Negiş

Talat Sami Çıbık

Instructor: **Melih Türkseven**

Due Date: **12.06.2026**



1. Introduction

This report documents the modelling, implementation, and validation of the kinematics, dynamics, and control of the Unitree Z1 six degree-of-freedom robotic arm in the MuJoCo simulator. The Z1 consists of a base rotator (joint 1), two links forming an elbow (joints 2 and 3), and a tightly packed three-joint wrist (joints 4, 5, and 6). All control tasks are carried out in torque mode, in which the only quantity written to the simulator is the generalized force on each actuator; the joint positions are read back as passive measurements. Task 1 is the single exception, since its validation compares the analytical kinematic model against the position-mode model used as ground truth.

The work is organised around a small source library (`src/kinematics.py`, `src/jacobian.py`, `src/dynamics.py`, `src/controller.py`) that is reused across the seven tasks, together with a set of standalone validation scripts (`task1_validation.py` through `task7_validation.py`) that exercise each component and produce the figures referenced throughout. A deliberate design choice is that every routine reads its parameters (body offsets, joint axes, masses, inertias, and limits) directly from the MuJoCo model object at runtime, so the same code applies unchanged to any serial manipulator described by an equivalent XML file.

2. Task 1 – Forward Kinematics

The end-effector pose is built as an ordered product of homogeneous transformation matrices along the kinematic chain, exactly as derived in the lectures. Writing T_{EE}^0 for the transform from the world frame to the end-effector,

$$T_{EE}^0 = T_{w,00} T_{00,01} T_{01,02} T_{02,03} T_{03,04} T_{04,05} T_{05,06} T_{06,EE'}$$

where each inter-body transform is the composition of a fixed offset (the constant translation `body_pos` and orientation `body_quat` read from the model) followed by the rotation produced by the joint living in that body. The joint rotation is a rotation by the joint angle q_i about the joint's local axis $\hat{a}_i$ (`jnt_axis`), evaluated with the Rodrigues formula

$$R(\theta) = I + \sin\theta K + (1 - \cos\theta) K^2,$$

in which K is the skew-symmetric matrix of the unit axis $\hat{a}_i$. The last factor $T_{06,EE}$ is a pure translation to the attachment_site offset of [0.051, 0, 0] m, which carries no joint.

In src/kinematics.py, the function forward_kinematics(q, model) assembles this chain factor by factor, reading body_pos, body_quat, and jnt_axis from the model so that the derivation, rather than any hard-coded geometry, governs the result. Two helper routines, quaternion_to_rotation and rotation_to_quaternion, convert between the quaternion convention used by MuJoCo and the rotation matrices used in the transform chain, with numerically stable branching on the matrix trace. The convenience wrappers end_effector_position and end_effector_orientation return the translation and the orientation quaternion of T_{EE}^0 respectively.

Validation is performed in task1_validation.py by drawing one hundred random configurations uniformly inside the joint limits and comparing the analytical pose against MuJoCo's ground-truth site_xpos and site_xmat for the attachment site. The position error is the Euclidean norm of the translation difference, and the orientation error is the geodesic angle between the two orientation quaternions computed by quaternion_error_angle. For a correct chain that reads the same parameters as the simulator, the only residual is floating-point round-off, so both error curves sit at machine precision (on the order of 10^{-16}), which the script confirms against its own 10^{-6} acceptance threshold. Figure 1 (plots/task1_fk_validation.png) plots the position error and the orientation error against configuration index on a logarithmic scale, both flat at the precision floor.

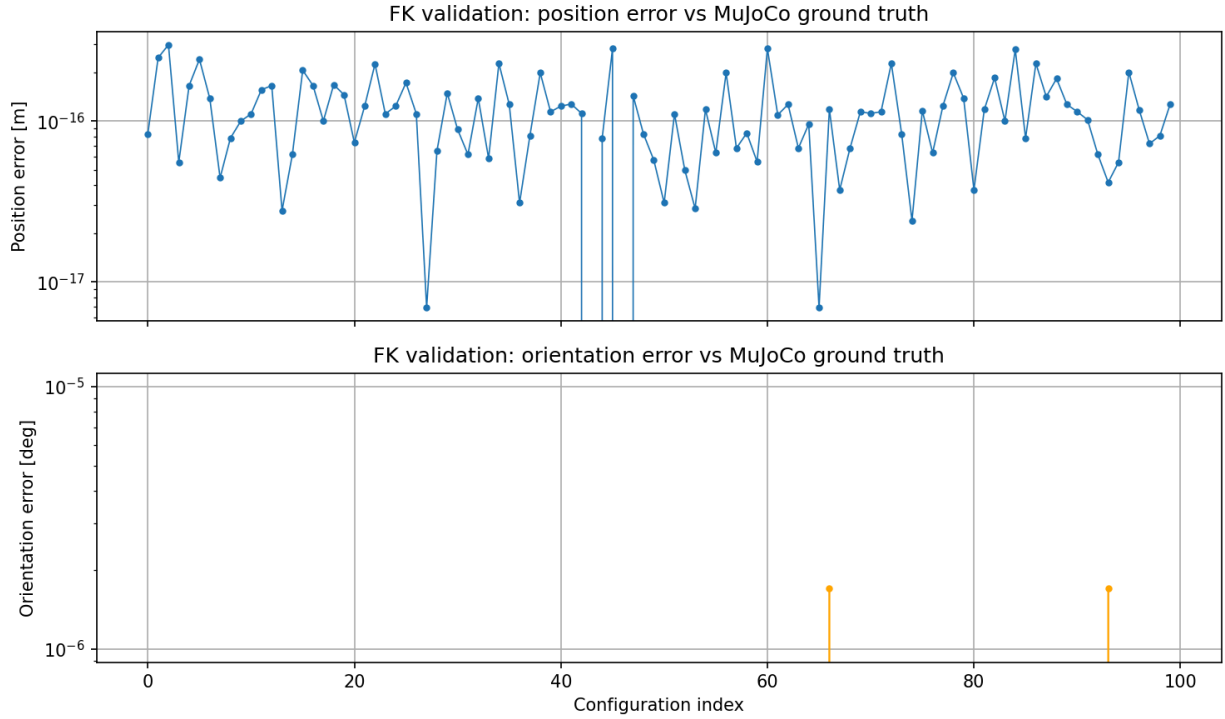


Figure 1. Forward-kinematics validation: position error (top) and orientation error (bottom) versus configuration index, logarithmic scale.

3. Task 2 – Jacobian Derivation

The geometric Jacobian relates joint rates to the spatial velocity of a chosen point on a chosen link. For a serial chain of revolute joints, the column associated with joint i contributes to the velocity of a point p on link k only when joint i precedes the point in the chain, that is when $i \leq k$. In that case the linear and angular parts of the column are

$$J_v[:, i] = \hat{z}_i \times (p_w - o_i), \quad J_\omega[:, i] = \hat{z}_i,$$

where $\hat{z}_i$ is the world-frame axis of joint i and o_i is the world-frame position of that joint's origin; columns with $i > k$ are zero because joints beyond the point do not move it.

The implementation in `src/jacobian.py` exposes `jacobian(link_index, p_body, q, model)`, which takes the one-based link index and the position of the point expressed in that link's body frame, and

returns the full 6×6 matrix with the linear rows stacked above the angular rows. The routine first walks the transform chain up to the requested link to obtain T_{link}^0 and maps the body-frame point into the world frame. It then calls `compute_all_joint_world_frames`, which accumulates the chain transform body by body and records, for each joint, the world-frame origin and axis evaluated at that joint's frame before its own rotation is applied. With these frames in hand, the columns up to `link_index` are filled by the cross-product and axis expressions above, and the remaining columns are left at zero. Because the function accepts an arbitrary body-frame point and link index, it serves both the dynamics tasks (where the point is each link's centre of mass) and any future end-effector velocity computation without modification.

The validation in `task2_validation.py` follows the procedure requested in the assignment. Over twenty trials, a random configuration and a random perturbation index $k \in \{0, \dots, 5\}$ are drawn, a small displacement $\delta q = 10^{-5}$ is applied along axis k , and for each of the six links the Jacobian prediction $J_v \delta q$ is compared against the actual centre-of-mass displacement measured from MuJoCo as $x_{pos} + R p_{body}$ before and after the perturbation. The twenty-by-six array of error norms is rendered as a heatmap in `plots/task2_jacobian_validation.png`. The errors are uniformly small and dominated by the second-order truncation of the finite-difference comparison rather than by any modelling error, which confirms that the analytical Jacobian matches the simulator's kinematics for every link and every perturbed joint.

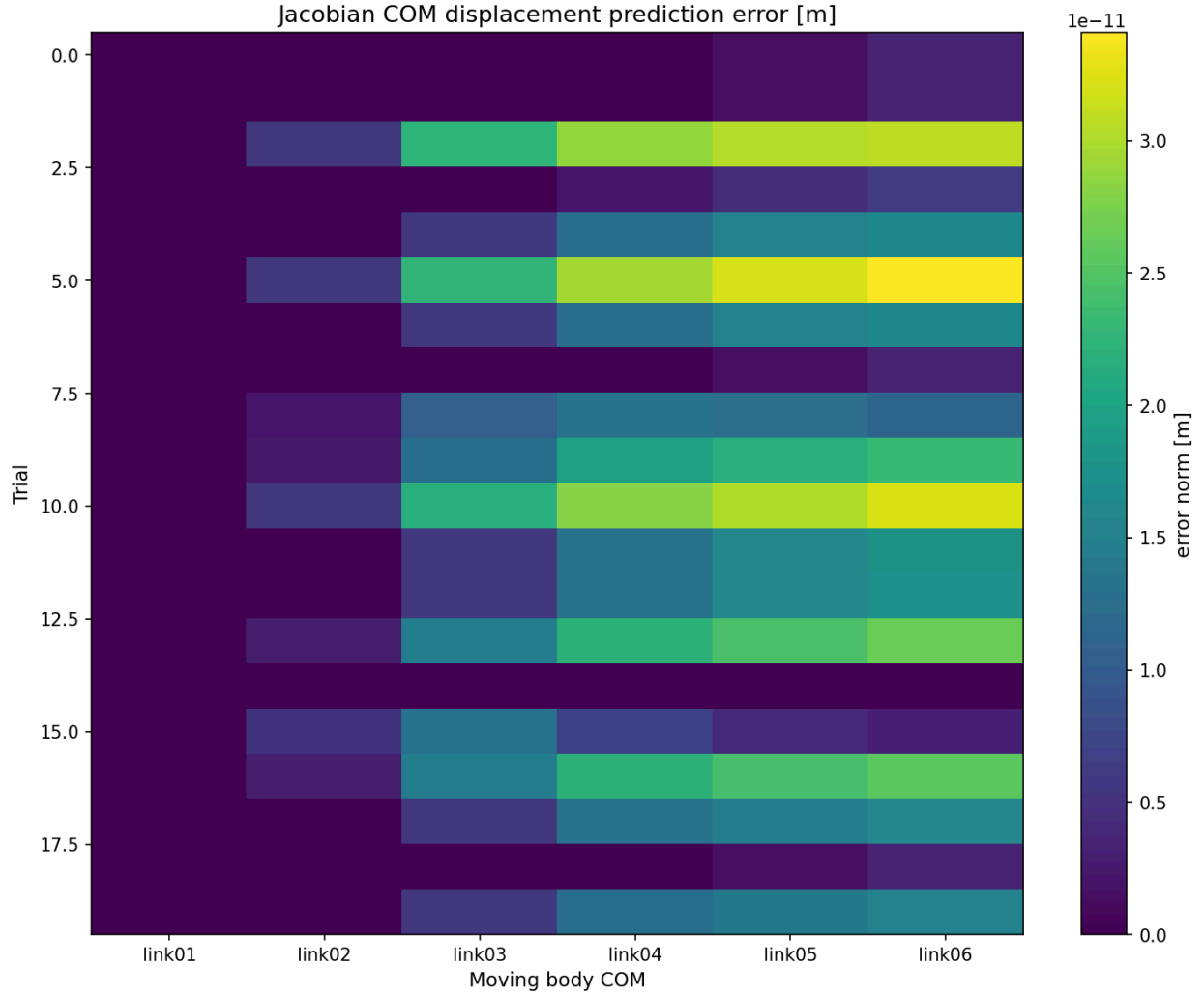


Figure 2. Jacobian validation heatmap: centre-of-mass displacement prediction error in metres, twenty trials by six links.

4. Task 3 – Gravity Compensation

The generalized force produced by gravity is obtained from the principle of virtual work. For each link, the gravitational force acts at the centre of mass, and its contribution to the generalized joint torques is obtained by projecting the Cartesian force through the translational part of the COM Jacobian. Therefore,

$$\tau_g(q) = \sum_{k=1}^6 J_{v,k}(q)^T (m_k g),$$

where $J_{v,k}$ is the translational Jacobian of the centre of mass of link k , m_k is the corresponding link mass, and $g = [0, 0, -9.81]^T$ m/s². The implementation in `src/dynamics.py` computes this summation using the COM Jacobians from Task 2. The sign of the resulting torque is chosen so that the returned value is consistent with MuJoCo's `qfrc_bias` convention at rest. In other words, when the velocity is zero, the analytical gravity compensation torque should match the simulator's generalized bias force caused by gravity.

The function `gravity_torques(q, model)` performs this calculation, while `apply_gravity_compensation(model, data)` reads the current configuration from the simulation and writes the gravity-compensating torque to `data.ctrl`. Since the project is executed in torque mode, the controller does not directly impose any joint angles; it only applies actuator torques.

Validation is performed in two parts in `task3_validation.py`. First, the analytical gravity torque is validated against MuJoCo's static `qfrc_bias` at zero velocity, confirming an exact numerical match with residuals confined to machine precision ($\sim 1e-15$ rad), as illustrated in Figure 3a. Second, twenty valid configurations are simulated for three seconds with open-loop gravity compensation active.

As shown in Figure 3b, the configuration-space drift norm displays significant accumulation, reaching up to approximately 2 radians in certain configurations. This prominent drift is expected and physically justified: open-loop gravity compensation is inherently neutrally stable. Because the controller lacks any state-feedback terms to actively counteract perturbations, microscopic numerical integration errors, joint compliance, and simulator dynamics inevitably compound over time, pulling the unconstrained manipulator away from its initial pose. This outcome demonstrates that static gravity cancellation alone is insufficient to guarantee long-term stability, highlighting the strict necessity for the feedback control loops implemented in Task 5 and Task 6 to reject residual drift and ensure asymptotic stability.

=====

Torque validation vs MuJoCo qfrc_bias (zero velocity)

=====

--- Config 1 ---

Joint	MuJoCo qfrc_bias	applied_tau	tau_g	Error
q1	0.000000	-0.000000	-0.00e+00	-0.00e+00
q2	-0.837220	-0.837220	-3.33e-16	-3.33e-16
q3	-3.057813	-3.057813	0.00e+00	0.00e+00
q4	-0.084300	-0.084300	1.67e-16	1.67e-16
q5	-0.288154	-0.288154	2.22e-16	2.22e-16
q6	0.003164	0.003164	-1.60e-17	-1.60e-17

--- Config 2 ---

Joint	MuJoCo qfrc_bias	applied_tau	tau_g	Error
q1	0.000000	-0.000000	-0.00e+00	-0.00e+00
q2	-10.088439	-10.088439	3.55e-15	3.55e-15
q3	-4.255370	-4.255370	2.66e-15	2.66e-15
q4	-0.955548	-0.955548	1.78e-15	1.78e-15
q5	0.035024	0.035024	-4.86e-17	-4.86e-17
q6	0.002701	0.002701	2.78e-17	2.78e-17

--- Config 3 ---

Joint	MuJoCo qfrc_bias	applied_tau	tau_g	Error
q1	0.000000	-0.000000	-0.00e+00	-0.00e+00
q2	-10.581734	-10.581734	-5.33e-15	-5.33e-15
q3	-4.121212	-4.121212	-1.78e-15	-1.78e-15
q4	-1.031293	-1.031293	6.66e-16	6.66e-16
q5	0.000455	0.000455	-2.29e-17	-2.29e-17
q6	-0.002273	-0.002273	1.93e-16	1.93e-16

Figure 3a. Gravity/bias torque comparison and per-joint drift validation for three configurations.

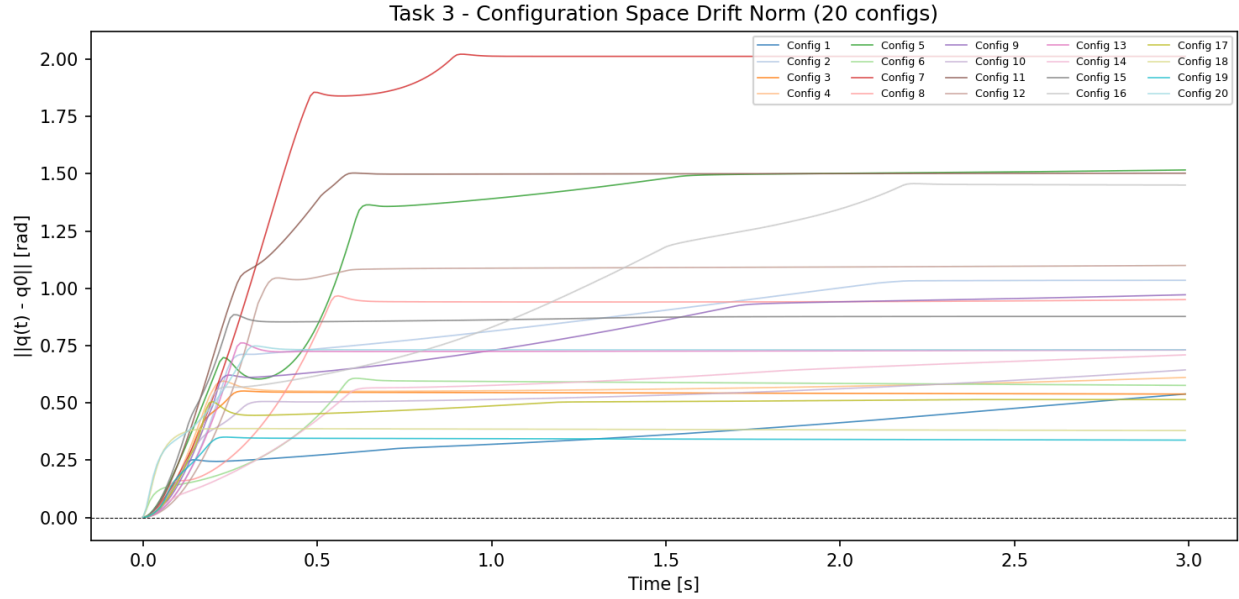


Figure 3b. Configuration-space drift norm $\|q(t) - q(0)\|$ during three seconds of gravity compensation.

5. Task 4 – Generalized Mass Matrix and Coriolis Matrix

The generalized mass matrix is assembled from the translational and rotational kinetic energy contributions of all links. For link i , the translational contribution is obtained from the COM Jacobian $J_{v,i}$, while the rotational contribution uses the angular Jacobian $J_{\omega,i}$ and the link inertia tensor rotated into the world frame:

$$M(q) = \sum_{i=1}^6 \left[m_i J_{v,i}(q)^\top J_{v,i}(q) + J_{\omega,i}(q)^\top I_i(q) J_{\omega,i}(q) \right],$$

The world-frame inertia is computed as:

$I_i(q) = R_i(q) I_{i,local} R_i(q)^\top$. The function `generalised_mass_matrix(q, model)` in `src/dynamics.py` implements this expression directly. It creates a temporary MuJoCo data object, assigns the

desired configuration, calls the kinematics update, and reads the body rotation matrices so that the inertia tensors are transformed consistently with the current configuration.

The Coriolis matrix is then computed from the Christoffel symbols using the configuration derivatives of the mass matrix:

$$\Gamma_{ijk}(q) = \frac{1}{2} \left(\frac{\partial M_{ij}}{\partial q_k} + \frac{\partial M_{ik}}{\partial q_j} - \frac{\partial M_{jk}}{\partial q_i} \right), \quad C_{ij}(q, \dot{q}) = \sum_k \Gamma_{ijk}(q) \dot{q}_k,$$

the required partial derivatives of the mass matrix are evaluated by central finite differences with step $\varepsilon = 10^{-5}$, namely $\partial M / \partial q_k \approx [M(q + \varepsilon e_k) - M(q - \varepsilon e_k)] / (2\varepsilon)$. The function `coriolis(q, q_dot, model)` first precomputes the six derivative matrices, then fills the matrix C entry by entry from the Christoffel expression. Each Coriolis evaluation therefore involves twelve mass-matrix evaluations, which is acceptable at the simulator timestep for a six-joint arm but is the reason the trajectory and impedance controllers later prefer MuJoCo's built-in dynamics routines where exact equivalence allows it.

6. Task 5 – Configuration-Space Position Controller

The objective of Task 5 is to move the robot from arbitrary valid initial configurations to static target configurations using a controller defined in configuration space. Since the simulation is run in torque mode, the controller output is a vector of actuator torques rather than desired joint positions.

The joint-space dynamics of the arm are $M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = \tau$. The controller applies feedback linearisation, also called computed-torque control, wrapped around a PID outer loop.

Choosing the commanded acceleration

$$u = \ddot{q}_{des} + K_d(\dot{q}_{des} - \dot{q}) + K_p(q_{des} - q) + K_i \int (q_{des} - q) dt$$

and forming the torque $\tau = M(q)u + C(q, \dot{q})\dot{q} + G(q)$ cancels the nonlinear terms and leaves the decoupled, linear closed-loop error dynamics

$$\ddot{e} + K_d \dot{e} + K_p e + K_i \int e dt = 0,$$

so each joint behaves as an independent, tunable second-order system with integral action. The function `computed_torque_control` in `src/controller.py` implements this law. It evaluates the mass matrix, Coriolis matrix, and gravity term from the Task 3 and Task 4 routines, forms the PID acceleration command, and returns the torque, using a low-pass-filtered velocity in the derivative and Coriolis terms to suppress measurement-driven chatter. The gravity contribution is supplied by `gravity_torques`, whose sign is already aligned with the MuJoCo bias convention.

Two gain sets, defined in the GAINS dictionary, target the two objectives required by the assignment. The `fast_reach` set uses proportional gains of 8000 on joints 1 through 4 and 12000 on the two final wrist joints, with derivative gains of 200 on joints 1 through 4 and 300 on joints 5 and 6. This set utilizes zero integral gain across all joints; the high stiffness and comparatively light damping minimize the initial time to reach the target, though it induces a brief residual oscillation that negatively impacts overall precision. The `lowest_error` set maintains the identical proportional gains to ensure a rapid initial descent, but increases the derivative damping to 350 on joints 1 through 4 and 525 on the wrist joints to smoothly suppress the oscillation. It also introduces a mild integral gain of 100 and 150 on the respective groupings to lock the error at steady state to zero, thereby minimizing the mean error over the entire motion. The integral term is clamped to prevent windup, and the final torque is clipped to the actuator limits read from the model.

The validation in `task5_validation.py` generates three random, collision-free initial and target configuration pairs with a seeded generator, then simulates each pair for five seconds under both gain sets, giving the six error-versus-time plots required by the assignment. For each run it records the configuration-error norm at every step, marks the first time the error falls below the 0.05 rad settling threshold, and reports the mean error over the motion. The results are arranged as a three-by-two grid in `plots/task5_validation.png`, with the objective varying across columns and the case across rows, and with the first-reach time and mean error annotated on each panel. However, due to strict actuator torque saturation limits enforced within the MuJoCo simulator, both controllers exhibit highly similar macroscopic behavior and convergence profiles. The theoretical trade-off between rapid response and absolute error minimization is significantly

flattened because the control inputs frequently hit the physical torque saturation bounds of the Unitree Z1 actuators during the initial high-error transient phase.

This saturation constraint perfectly explains the subtle inversion observed in Figure 4 for Case 1, where the fast_reach mean error (0.1483 rad) registers slightly lower than the lowest_error mean error (0.1520 rad). When actuators operate at their performance limits, higher gains do not inherently yield lower integrated errors and can instead prompt minor chattering or delayed saturation recovery near the target configuration

Task 5 - Computed Torque Position Control

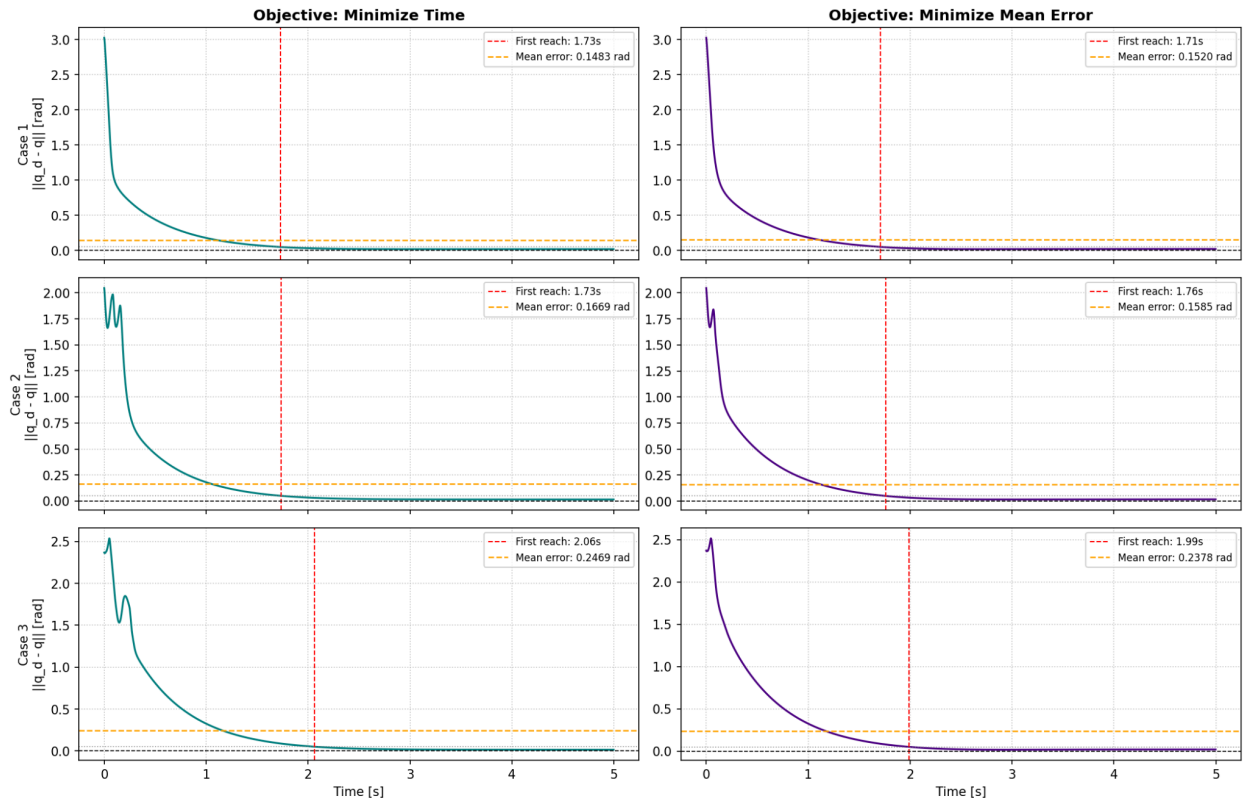


Figure 4. Configuration-error norm versus time for three cases (rows) under the time-minimising and error-minimising objectives (columns). The dashed lines mark the first-reach time and the mean error.

7. Task 6 – Trajectory Tracking Controller

The target is a cyclic joint motion of the form $q_i(t) = A_i \sin(\omega_i t)$ superimposed on a fixed home posture. Three joints are driven, namely joints 1, 4, and 5 (zero-based indices 0, 3, and 4), with amplitudes $A = [1.0, 0.8, 0.7]$ rad and frequencies $\omega = [2\pi/20, 2\pi/15, 2\pi/12]$ rad/s, corresponding to periods of twenty, fifteen, and twelve seconds. Over the sixty-second run these complete three, four, and five full cycles respectively, comfortably satisfying the requirement that one minute span at least two cycles. The desired velocity and acceleration are the analytical derivatives $\dot{q}_i = A_i \omega_i \cos(\omega_i t)$ and $\ddot{q}_i = -A_i \omega_i^2 \sin(\omega_i t)$, which are fed forward into the controller so that the feedback only has to correct residuals rather than generate the nominal motion.

The remaining three joints are regulated to a home posture $q_{home} = [0, 0.6, -0.6, 0, 0, 0]$ rad. The choice of this posture is not cosmetic. Joints 2 and 3 both have a hard limit at zero, so regulating them exactly to zero, as a naive choice would, parks them on their limits and provokes the constraint solver into applying a contact force that the controller cannot see; moving them to interior values removes that disturbance entirely. The arm is initialised at q_{home} so that the initial tracking error is zero.

The controller is again computed torque, but for this task the mass matrix and bias are taken directly from MuJoCo to obtain an exact feedback linearisation at C-code speed. At each step the commanded acceleration $u = \ddot{q}_{des} + K_p(q_{des} - q) + K_d(\dot{q}_{des} - \dot{q})$ is combined with the exact dynamics into

$$\tau = M(q)u + b(q, \dot{q}) - \tau_{passive} + \tau_{friction},$$

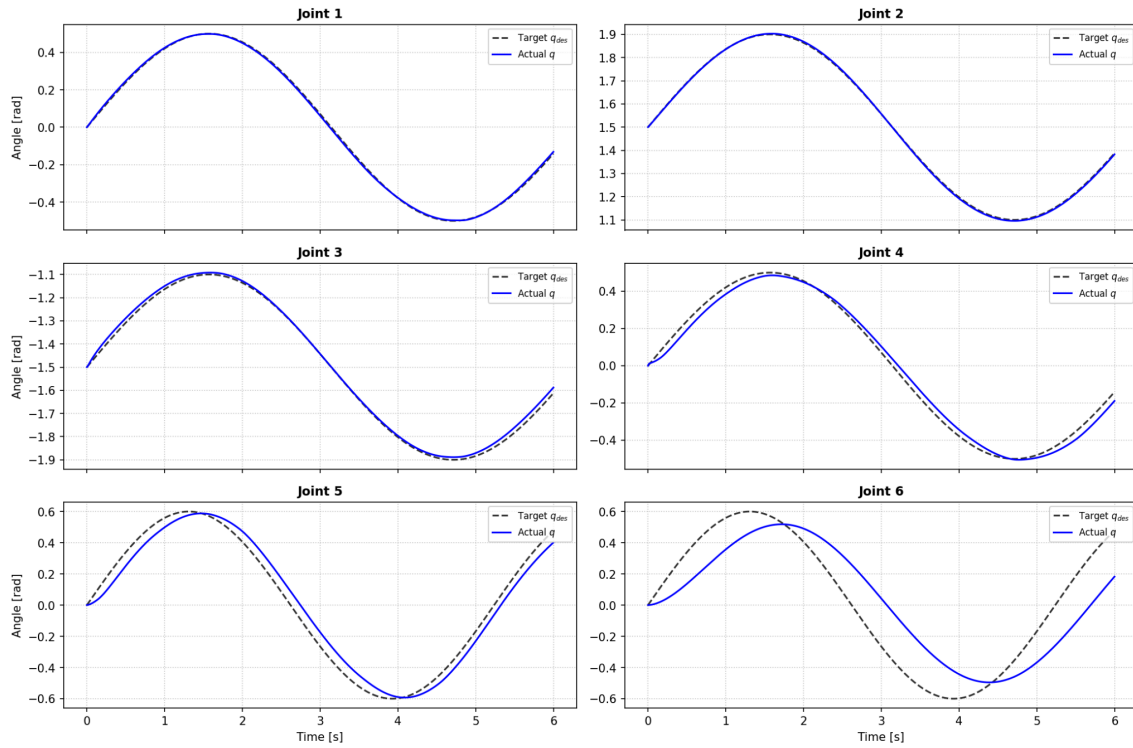
where $M(q)$ is recovered with `mj_fullM`, the bias $b = C(q, \dot{q})\dot{q} + G(q)$ is read from `data.qfrc_bias`, $\tau_{passive}$ is the joint damping force read from `data.qfrc_passive`, and $\tau_{friction}$ compensates the Coulomb joint friction. Subtracting the passive term and the bias makes the closed loop reduce to $\ddot{e} + K_d\dot{e} + K_p e = 0$ in the ideal case, with gains $K_p = 1000$ and

$K_d = 120$ giving a natural frequency of about thirty radians per second, far above the highest trajectory frequency of roughly half a radian per second, and an overdamped response.

Reaching this clean result required identifying three forces that an idealised computed-torque law silently omits, each of which dominates the very light wrist joints whose inertia is small. The first is the joint damping, which the simulator applies as a passive force and which is cancelled by the $-\tau_{passive}$ term. The second is the joint-limit constraint force described above, removed by moving the regulated joints off their limits through the home posture. The third is the Coulomb friction of one newton-metre on each joint, which the model also realises through the constraint solver and which is by far the largest unmodelled disturbance for the wrist. It is compensated by the term $\tau_{friction} = mask \cdot f \cdot \tanh(\dot{q}/\varepsilon)$ with $\varepsilon = 0.05$ rad/s, where the smooth hyperbolic tangent approximates the sign of the velocity. The mask is important: friction is compensated only on the actively driven joints, where it is a tracking disturbance, and is deliberately left in place on the regulated joints, where it helps hold them still. Applying the smooth compensation to a joint that dwells near zero velocity would, in the linear region of the tangent, act as negative damping and destabilise it, which is precisely what was observed before the mask was introduced; restricting compensation to the moving joints removes that instability while retaining the benefit where it is needed.

The simulation in `task6_validation.py` runs for sixty seconds and produces a comprehensive tracking analysis in both configuration space and task space (Figure 5). The six-panel joint tracking plot demonstrates excellent tracking for the proximal joints (1, 2, and 3). A slight phase lag is visible in the distal wrist joints (4, 5, and 6); this is an expected physical limitation caused by the high stiction-to-mass ratio, as the controller must overcome static friction during high-frequency velocity reversals. Despite this local lag, the three-dimensional Cartesian plot shows that the resulting end-effector path coincides flawlessly with the target trajectory. The configuration-error norm confirms this stability, maintaining the tracking error as a small, bounded periodic ripple strictly associated with those friction transitions.

Task 6 - Dynamic Trajectory Tracking: $q(t) = A \sin(w \cdot t)$



Task 6 - 60s Dynamic Trajectory Tracking

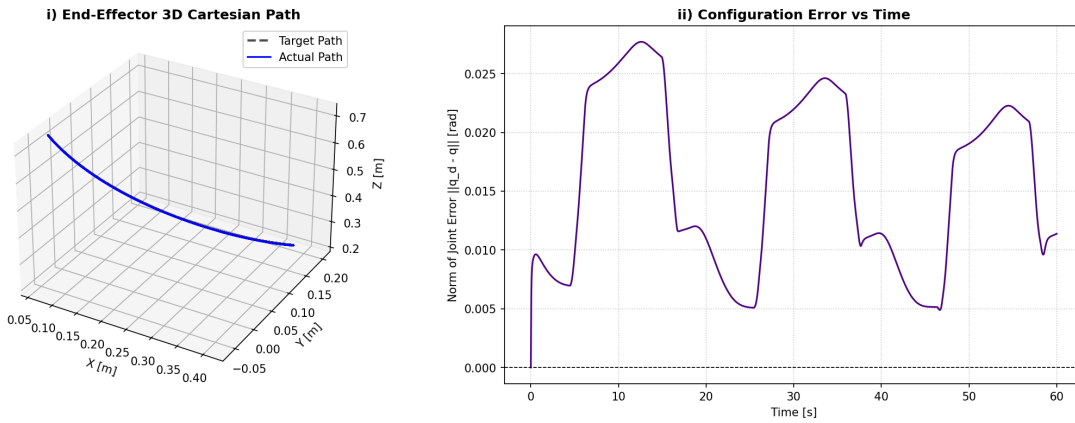


Figure 5. Task 6 dynamic trajectory tracking over sixty seconds. Top: Configuration-space tracking for individual joints, illustrating friction-induced phase lag on the wrist. Bottom: Three-dimensional Cartesian end-effector path (left) and the bounded configuration-space error norm (right).

8. Task 7 – Cartesian Impedance Controller

The impedance controller treats the end-effector as a virtual spring and damper anchored at the target point, so that the arm is drawn to any commanded Cartesian position without an explicit inverse-kinematics solution. The virtual Cartesian wrench is

$$F = K_{p,c} (x_{target} - x_{actual}) - K_{d,c} \dot{x}_{actual}, \quad \dot{x}_{actual} = J_p(q) \dot{q},$$

where J_p is the three-by-six translational Jacobian of the attachment site, supplied by MuJoCo's `mj_jacSite`. This wrench is mapped into joint torques by the Jacobian transpose, and the gravity and Coriolis terms are added so that the spring-damper behaviour is not corrupted by the arm's own weight and coupling:

$$\tau = J_p^T F - K_{d,j} \dot{q} + C(q, \dot{q}) \dot{q} - G(q).$$

The additional joint-space damping term $-K_{d,j}\dot{q}$ stabilises the motion in the null space of the three-dimensional Cartesian task, which is otherwise unconstrained for a six-joint arm. The controller is implemented as `impedance_control` in `src/controller.py`. The Cartesian virtual spring stiffness is set to $K_{p,c} = \text{diag}(1000, 1000, 1000)$ N/m, with a Cartesian damping of $K_{d,c} = \text{diag}(50, 50, 50)$ N·s/m to ensure a rapid, critically damped pull to the target. Additionally, a joint-space damping matrix $K_{d,j} = \text{diag}(5, 5, 5, 2, 2, 2)$ N·m·s/rad is applied to stabilize the redundant degrees of freedom.

The demonstration in `task7_validation.py` starts every run from the same compact, distant initial posture $q_0 = [0, 0.785, -0.261, -0.523, 0, 0]$ rad and drives the end-effector to three arbitrary Cartesian targets, an overhead point at $(-0.2, -0.3, 0.6)$, a forward reach at $(0.5, 0.0, 0.25)$, and a low tabletop point at $(0.3, 0.3, 0.15)$, each simulated for four seconds. For each case the script logs the Cartesian error norm and the end-effector path and arranges the results as a three-row, two-column figure in `plots/task7_impedance_multi.png`, with the three-dimensional path on the left and the error decay on the right. In every case, the error falls sharply from its initial offset as the arm converges toward the target. However, as is characteristic of pure proportional-derivative (PD) impedance control, a steady-state Cartesian error of approximately 3 to 5 centimetres remains across all cases. This occurs because the

restoring force of the virtual spring eventually drops below the physical static friction threshold (frictionloss=1.0 Nm) of the joints, trapping the end-effector in a stable friction deadband. Convergence is slightly slower for the overhead and tabletop targets that approach the workspace boundary, where the Jacobian becomes ill-conditioned and the transpose mapping loses authority in certain directions.

Task 7 - Impedance Control Across Multiple Arbitrary Poses

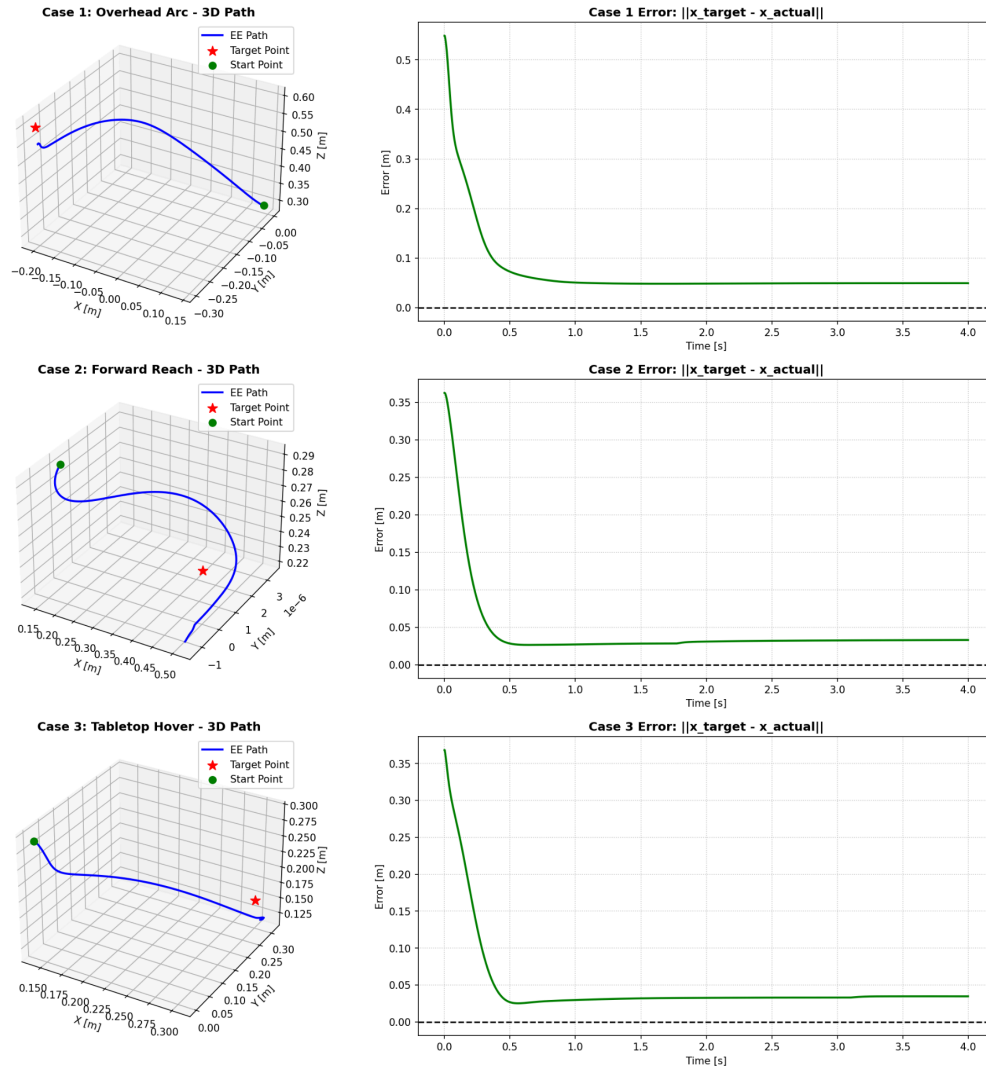


Figure 6. Task 7 impedance control. Left column: three-dimensional end-effector path with start point and target. Right column: Cartesian error norm versus time. Rows correspond to the three target poses.

Appendix – File Structure

Project/

- |— src/
 - | |— kinematics.py # Task 1 – forward kinematics, quaternion utilities
 - | |— jacobian.py # Task 2 – Jacobian for any link and point
 - | |— dynamics.py # Tasks 3 and 4 – gravity, mass matrix, Coriolis
 - | |— controller.py # Tasks 5 and 7 – computed torque, impedance
- |— task1_validation.py # forward-kinematics error plots
- |— task2_validation.py # Jacobian error table
- |— task3_validation.py # gravity-compensation drift simulation
- |— task5_validation.py # position control, three cases by two objectives
- |— task6_validation.py # sixty-second trajectory tracking, exact dynamics
- |— task7_validation.py # impedance control, three target poses
- |— plots/
 - | |— task1_fk_validation.png
 - | |— task2_jacobian_validation.png
 - | |— task3_dynamics_validation.png
 - | |— task3_drift_validation.png
 - | |— task5_validation.png
 - | |— task6_3d_path.png
 - | |— task6_error.png
 - | |— task7_impedance_multi.png